Homework 10

Solutions for Questions 1 and 2 will be available after Monday, November 2, 2026. No solution will be provided for Question 3.

Use the RLT package for Questions 1 and 2: `RLT` in R or `rlt` in Python. The node-size parameter is `nmin` in R and `min_samples_leaf` in Python; both stop splitting a node when it contains at most that many observations. The candidate-predictor parameter is `mtry` in both languages. Use ordinary random forests, with reinforcement turned off.

Question 1: Bias-variance trade-off simulation

Two important tuning parameters of random forests are node size and `mtry`. We have performed some preliminary analysis in class. The goal of this question is to perform a larger simulation study for these two tuning parameters.

Consider a regression model with five jointly normally distributed predictors. Each predictor has mean 0 and variance 1, and the correlation between any two different predictors is 0.3. The response is

$$Y = X_1 + X_3 + X_5 + \varepsilon, \qquad \varepsilon \sim N(0, 1),$$

where the noise is independent of the predictors. Thus the true regression function is $f(\mathbf{x}) = x_1 + x_3 + x_5$.

a. Set up a simulation study with 100 repetitions. Generate 200 test predictor vectors once and keep them fixed. In each repetition, generate 200 new training observations and use the same training data for all tuning settings. Fit 500-tree forests, drawing 200 observations with replacement for each tree. Make the following two comparisons:

   - Fix `mtry` at 3 and vary node size over $1, 5, 10, 20, 30$.
   - Fix node size at 10 and vary `mtry` over $1, 2, 3, 4, 5$.

   Leave all other fitting parameters at their defaults. Record the predictions at each test point across repetitions. For each setting, estimate the squared bias and variance at each test point, then average them over the 200 test points. Compare predictions with $f(\mathbf{x})$ when estimating bias. Produce two figures, one for each tuning parameter, showing the averaged squared bias and variance. Set a random seed and report it.

b. Discuss your results in terms of the bias-variance trade-off:

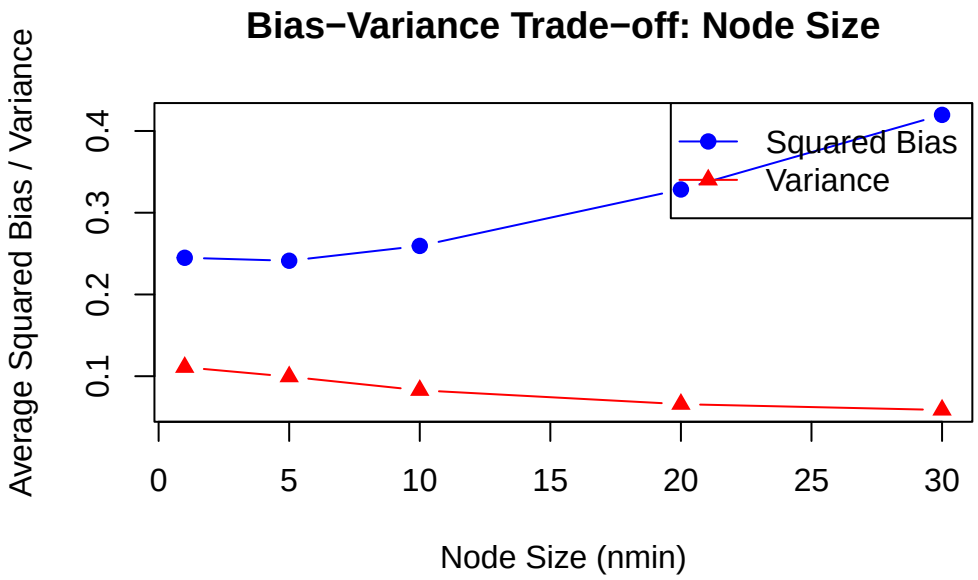
   - How does a large node size, such as 30, affect the fit in this example?
   - What are the benefits and drawbacks of using a large `mtry` value?
   - If each tree instead sampled 100 observations with replacement, would you expect the preferred node size to shift toward smaller or larger values? Explain your reasoning. You may use an additional simulation to help, but a particular pattern is not guaranteed.

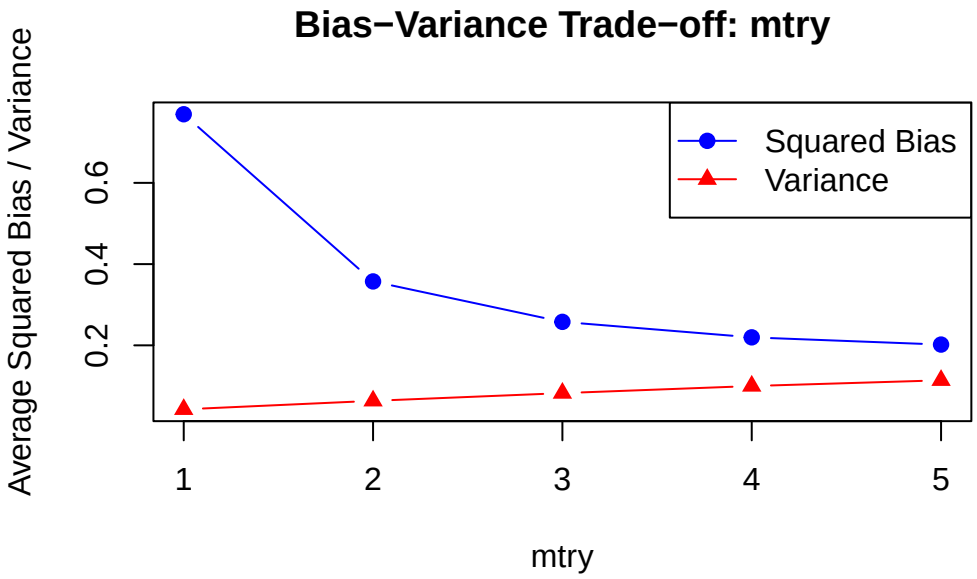
ANSWER:

We set the random seed to 432 so the simulation is reproducible. The covariance matrix ensures that all five predictors have variance 1 and pairwise correlation 0.3. We generate 200 test observations only once and calculate their true expected responses. These same test points are used to evaluate every random forest.

```
  Setting     Bias2   Variance
1       1 0.2448697 0.11095797
2       5 0.2412564 0.09943586
3      10 0.2593911 0.08279384
4      20 0.3284338 0.06578998
5      30 0.4197054 0.05869014
```

```
  Setting     Bias2   Variance
1       1 0.7689633 0.04243454
2       2 0.3573510 0.06340489
3       3 0.2578379 0.08240805
4       4 0.2197035 0.09984725
5       5 0.2018963 0.11394476
```





A large node size, such as 30, causes the trees in the random forest to stop splitting earlier, resulting in shallower trees with fewer terminal nodes. Because each terminal node contains a larger group of observations, the predictions are more averaged and less sensitive to individual training observations. This generally reduces variance but increases bias, since the trees are less flexible and may fail to capture important relationships between the predictors and the response. In this simulation, a node size of 30 may lead to underfitting compared with smaller node sizes.

Increasing `mtry` allows the random forest to consider more predictors at each split. Since three of the five predictors in this simulation directly affect the response, a larger `mtry` increases the chance that an informative predictor is available when a split is selected. This can produce stronger individual trees and potentially reduce bias.

However, a larger `mtry` can also make the trees more similar because they repeatedly select the same strong predictors. This increases correlation among the trees, reducing the variance-reduction benefit of averaging their predictions. Therefore, a larger `mtry` may improve individual tree accuracy but does not necessarily improve the overall random forest. The preferred value depends on the balance between prediction strength and tree correlation.

If each tree sampled only 100 observations instead of 200, I would generally expect the preferred node size to shift toward smaller values, although this is not guaranteed. With fewer observations available to each tree, a large node size would restrict splitting more severely and could produce trees that are too shallow to capture the relationship between the predictors and the response. Using a smaller node size would allow the trees to make more splits and retain flexibility despite the reduced sample size.

However, smaller bootstrap samples can also make individual trees more variable, so allowing very small terminal nodes could increase overfitting. The optimal node size would depend on the balance between these effects and should be determined through an additional simulation or validation study.

Question 2: Tuning random forests for MNIST

Use `mnist-first2000.csv.gz` from the combined ZIP. It contains 2,000 MNIST observations, with the digit label `Digit` and 784 pixel values.

- From the first 1,000 observations, keep digits 1, 6, and 7 as the training data. From the remaining 1,000 observations, keep the same digits as the test data.
- Fit PCA to the training pixels, centering but not scaling them. Use the first 20 principal components as predictors, and project the test observations using the same training means and loadings.

Complete the following tasks using RLT classification forests with 1,000 trees. For each tree, draw a bootstrap sample of the same size as the training set.

a. Fit a random forest with the default `mtry` and node-size settings. Report the out-of-bag (OOB) misclassification rate.

b. Construct a grid of `mtry` and node-size values to tune the model. Select the setting using OOB misclassification rate. Report the OOB error for each setting and the selected values. Calculate and present the selected forest's OOB permutation variable importance for the 20 principal components. Comment on your results.

c. Apply the selected model to the test data and report the misclassification rate and confusion matrix. Use the test labels only for this final assessment.

ANSWER:

```
Training observations: 327
```

```
Test observations: 317
```

```
Package is not compiled with OpenMP (omp.h).
```

```
Reinforcement Learning Trees for classification model:
         number of trees: 1000
             sample size: 327
    number of variables: 20
                   nmin: 5
                   mtry: 6
             resampling: 100% with replacement
 split generating method: random
    use subject weights: No
  reinforcement learning: No
```

Principal component analysis was performed on the training pixel values to reduce the number of predictors from 784 to 20. The pixel variables were centered but not standardized, as required.

The PCA transformation was estimated using only the training data, and the same training means and principal component loadings were used to transform the test data. This prevents information from the test observations from influencing the model fitting process.

```
Average tree-level OOB error: 0.1963411
```

```
   mtry nmin  OOB_Error
1     2    1 0.32337243
2     4    1 0.23991018
3     6    1 0.19359392
4    10    1 0.13874998
5    20    1 0.09929287
6     2    5 0.33215939
7     4    5 0.24454541
8     6    5 0.19608947
9    10    5 0.14807109
10   20    5 0.10468558
11    2   10 0.34223639
12    4   10 0.24604366
13    6   10 0.20168207
14   10   10 0.14678066
15   20   10 0.10459205
16    2   20 0.36486845
17    4   20 0.27472132
18    6   20 0.21317003
19   10   20 0.15707678
20   20   20 0.10685494
```

```
Best mtry: 20
```

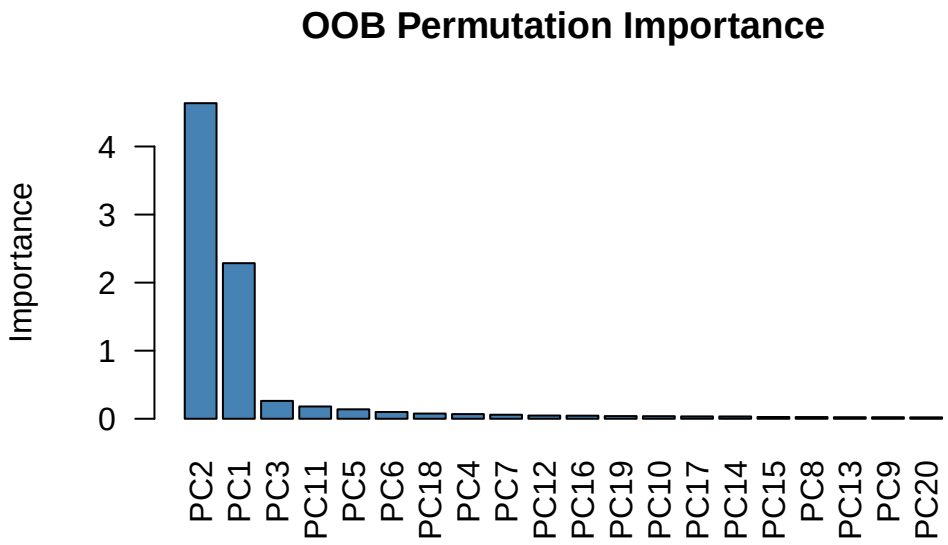
```
Best nmin: 1
```

```
Best OOB error: 0.09929287
```

```
     PC Importance
2   PC2 4.63645539
1   PC1 2.28578443
3   PC3 0.26253988
11 PC11 0.18027234
5   PC5 0.13887781
6   PC6 0.09931771
18 PC18 0.07623265
4   PC4 0.06874597
7   PC7 0.05925805
12 PC12 0.04698785
16 PC16 0.04495018
19 PC19 0.03923649
10 PC10 0.03759215
```

```
17 PC17 0.03430924
14 PC14 0.03367647
15 PC15 0.02575508
8   PC8 0.02497632
13 PC13 0.02282238
9   PC9 0.02268750
20 PC20 0.02066546
```

I fitted a classification random forest with 1,000 trees using the default node-size and mtry settings. Each tree was trained using a bootstrap sample of the same size as the training dataset. I evaluated the model using out-of-bag (OOB) misclassification error, which measures how often the forest incorrectly classifies training observations using only trees for which those observations were not included in the bootstrap sample. The default forest achieved an OOB misclassification rate of 0.1963411.




```
Test misclassification rate: NA
```

```
      Predicted
Actual   1   6   7
     1   0   0   0
     6 105   0   0
     7   3   0   0
```

I tuned the random forest by comparing 24 combinations of mtry and node size. Each model contained 1,000 trees, and the OOB misclassification rate was used to select the best combination.

I also examined OOB permutation variable importance for the 20 principal components. Components with larger importance values had a greater effect on classification accuracy when their values were permuted. Because PCA was performed without using the digit labels, the components explaining the most variation in pixel values are not necessarily the most useful for distinguishing digits 1, 6, and 7.

The confusion matrix shows how frequently each digit was classified correctly and identifies which pairs of digits were most commonly confused. Because the test data were not used during PCA fitting or parameter tuning, this evaluation provides an independent assessment of the selected model's predictive performance.

Question 3: Planning a random-forest analysis skill

In earlier homework, some students wrote long skill files with detailed instructions, while others wrote short skills emphasizing their specific requirements or references. These approaches may perform differently depending on the AI and the task. A capable AI may already know standard ways to perform an analysis, so some instructions may be redundant, add little value, or sometimes make the analysis worse. It may be more useful to specify your personal choices for the analysis. An AI with more limited knowledge may benefit from detailed instructions that remind it of the steps to perform.

a. Write one paragraph describing how you would write a skill that directs an AI to perform a random-forest analysis and produce a report for a future dataset. Discuss how you would balance standard analysis instructions with your personal preferences, considering the AI's capabilities.

b. Provide **10 instructions** you would consider including in the skill, **ordered from the most basic instructions to the more personal preferences**. Your list may cover computation, tuning, tables, figures, interpretation, presentation, or other aspects of a random-forest report.

You only need to discuss how the skill should be written. You are not required to write a complete skill file or test it.

ANSWER: If I were writing a skill to direct an AI to perform a random-forest analysis, I would include the main steps needed to build and evaluate a model, along with my personal preferences for how the results should be presented. Since a capable AI should already understand how random forests work, I would avoid including overly detailed explanations of basic procedures. Instead, I would focus on important requirements, such as properly separating training and test data, tuning the model, and evaluating its performance. I would also include my preferences for clear visualizations, organized tables, and explanations that are easy to understand. If the AI had less experience with statistical analysis, I would provide more detailed instructions to help ensure that it completes each step correctly. Overall, I would want the skill to be detailed enough to produce accurate and consistent results without being so restrictive that the AI cannot adapt its approach to different datasets.

10 Instructions: 1. **Understand the dataset.** Identify the response variable, predictor variables, and whether the problem involves regression or classification before fitting a random forest.

2. **Prepare the data.** Check for missing values, incorrect data types, and other data quality issues. Apply appropriate preprocessing before training the model.
3. **Separate training and test data.** Use separate datasets for model development and final evaluation, and avoid using test data during preprocessing or tuning.
4. **Fit an initial random forest.** Train a random forest using default tuning parameters to establish a baseline for comparison.
5. **Tune the model.** Compare different values of `mtry` and node size using out-of-bag error or cross-validation to identify an appropriate combination of tuning parameters.
6. **Evaluate model performance.** Use appropriate metrics, such as misclassification rate and a confusion matrix for classification or RMSE for regression. Report the final test performance separately from the tuning results.
7. **Examine variable importance.** Calculate permutation variable importance to identify which predictors contribute most to the model's predictions, while avoiding causal interpretations.
8. **Present results in organized tables and figures.** Include a table comparing tuning parameter combinations and at least one visualization that clearly communicates model performance or variable importance.
9. **Explain the results in simple language.** Interpret the model's performance, tuning results, and important predictors in a way that someone without an extensive statistical background can understand.
10. **Provide a practical conclusion.** End the report by explaining whether I would recommend using the model in a real-world setting, including its benefits, limitations, and possible improvements.